

Computer Laboratory - III

B.E. Semester II

Theory Concepts & Explanations

Department of Artificial Intelligence and Data Science
DYPIEMR, Pune

Practical No.	Topic
Practical 1	RPC - Remote Procedure Call
Practical 2	RMI - Remote Method Invocation
Practical 3	MapReduce under Hadoop
Practical 4	Fuzzy Sets and Operations
Practical 5	Genetic Algorithm + Neural Network
Practical 6	Clonal Selection Algorithm
Practical 7	Artificial Immune Pattern Recognition
Practical 8	DEAP - Distributed Evolutionary Algorithms
Practical 9	Hotel Booking using Java RMI
Practical 10	MapReduce for Weather Data Analysis

Practical 1: Remote Procedure Call (RPC)

What is RPC?

Remote Procedure Call (RPC) is an interprocess communication technique that allows a computer program to cause a procedure or subroutine to execute in a different address space (commonly on another computer on a shared network). It is coded as a normal procedure call without the programmer specifically coding the details for the remote interaction.

Types of RPC

1. Callback RPC

Enables a P2P paradigm between participating processes. Helps a process to be both client and server.

- Remotely processed interactive application problems
- Offers server with clients handle
- Callback makes the client process wait
- Manage callback deadlocks
- Facilitates a peer-to-peer paradigm among participating processes

2. Broadcast RPC

A client's request is broadcast on the network, processed by all servers which have the method for processing that request.

- Allows you to specify that the client's request message has to be broadcasted
- You can declare broadcast ports
- Helps to reduce the load on the physical network

3. Batch-mode RPC

Helps to queue separate RPC requests in a transmission buffer on the client-side, and then send them on a network in one batch to the server.

- Minimizes overhead by sending requests over the network in one batch
- Only efficient for applications that need lower call rates
- Needs a reliable transmission protocol

RPC Architecture Components

Component	Description
Client	Initiates the remote procedure call
Client Stub	Acts as gateway on client side, marshals parameters
RPC Runtime	Manages transmission, retransmission, routing, encryption
Server Stub	Acts as gateway on server side, unmarshals parameters

Server	Executes the actual procedure and returns result
--------	--

How RPC Works - Step by Step

- Step 1: Client, client stub, and one instance of RPC runtime execute on the client machine.
- Step 2: Client starts a client stub process by passing parameters in the usual way.
- Step 3: RPC Runtime manages the transmission of messages between the network across client and server.
- Step 4: After completing the server procedure, it returns to the server stub which marshalls the return values.
- Step 5: The transport layer sends back the result message to the client transport layer.
- Step 6: The client stub demarshalls (unpack) the return parameters and returns to the caller.

Advantages of RPC

- RPC method helps clients to communicate with servers using conventional procedure calls
- Supports process and thread-oriented models
- Makes the internal message passing mechanism hidden from the user
- Can be used for distributed and local environments
- Provides abstraction - network communication remains hidden from user

Disadvantages of RPC

- Passes parameters by values only - pointer values not allowed
- Remote procedure calling time can be significantly slower than local
- Highly vulnerable to failure as it involves communication system
- Not standardized - can be implemented in different ways
- Not flexible for hardware architecture - mostly interaction-based
- Cost of process is increased because of remote procedure call

Practical 2: Remote Method Invocation (RMI)

What is RMI?

The RMI (Remote Method Invocation) is an API that provides a mechanism to create distributed application in Java. The RMI allows an object to invoke methods on an object running in another JVM. The RMI provides remote communication between the applications using two objects: stub and skeleton.

Stub and Skeleton

Stub

The stub is an object that acts as a gateway for the client side. All outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes method on the stub object:

- It initiates a connection with remote Virtual Machine (JVM)
- It writes and transmits (marshals) the parameters to the remote JVM
- It waits for the result
- It reads (unmarshals) the return value or exception
- It finally returns the value to the caller

Skeleton

The skeleton is an object that acts as a gateway for the server side object. All incoming requests are routed through it. When the skeleton receives the incoming request:

- It reads the parameter for the remote method
- It invokes the method on the actual remote object
- It writes and transmits (marshals) the result to the caller

6 Steps to Write RMI Program

Step	Action
Step 1	Create the remote interface
Step 2	Provide the implementation of the remote interface
Step 3	Compile the implementation class and create stub and skeleton objects using rmic tool
Step 4	Start the registry service by rmiregistry tool
Step 5	Create and start the remote application
Step 6	Create and start the client application

Requirements for Distributed Applications

- The application needs to locate the remote method
- It needs to provide the communication with the remote objects

- The application needs to load the class definitions for the objects

The RMI application has all these features, so it is called a distributed application.

Practical 3: MapReduce under Hadoop

What is MapReduce?

MapReduce is a programming model and framework within the Hadoop ecosystem that enables efficient processing of big data by automatically distributing and parallelizing the computation. It consists of two fundamental tasks: Map and Reduce.

The 5 Phases of MapReduce

Phase	Description
1. Input Splits	MapReduce splits the input into smaller chunks. Each split is processed by one mapper task.
2. Mapping	Input data is processed and divided into smaller segments. Mapper transforms data into key-value pairs.
3. Shuffling	Output of mapper is passed to reducer by removing duplicates and grouping values. Can be parallelized.
4. Sorting	Intermediate key-value pairs are sorted by key before starting reducer phase. Happens simultaneously with shuffling.
5. Reducing	Intermediate values are reduced to produce a single output value. Final output stored on HDFS.

Advantages of MapReduce

- Can define mapper and reducer in several different languages using Hadoop streaming
- Facilitates automatic parallelization and distribution, reducing processing time
- Provides fault tolerance by re-executing failed map or reducer tasks
- Processing of data using MapReduce is a cost-effective solution
- Processes large volumes of unstructured data very quickly
- Ensures data security using HDFS and HBase security
- Uses a simple programming model to handle tasks efficiently

Limitations of MapReduce

- Low-level programming model which involves a lot of writing code
- Batch-based processing nature makes it unsuitable for real-time processing
- Does not support data pipelining or overlapping of Map and Reduce phases
- Task initialization, coordination, monitoring and scheduling take up large execution time
- Cannot cache intermediate data in memory, diminishing Hadoop's performance

How MapReduce Works - Word Count Example

Input Text: "This is an apple. Apple is red in color."

Phase	Output
Input Split 1	This is an apple
Input Split 2	Apple is red in color

Mapper 1 Output	This:1, is:1, an:1, apple:1
Mapper 2 Output	Apple:1, is:1, red:1, in:1, color:1
After Shuffle	is:[1,1], apple:[1,1], This:[1], an:[1], red:[1], in:[1], color:[1]
Final Output	This:1, is:2, an:1, apple:2, red:1, in:1, color:1

Practical 4: Fuzzy Sets and Operations

What is a Fuzzy Set?

Fuzzy refers to something that is unclear or vague. Hence, a Fuzzy Set is a set where every key is associated with a value between 0 to 1 based on certainty. This value is often called the degree of membership. Unlike crisp sets where an element either belongs (1) or does not belong (0), fuzzy sets allow partial membership.

Degree of Membership

The degree of membership represents how much an element belongs to a fuzzy set:

Degree	Meaning
0	Element does NOT belong to the set
0.5	Element PARTIALLY belongs (50%)
1	Element FULLY belongs to the set
0.3	Element belongs only 30% to the set

4 Operations on Fuzzy Sets

Operation	Formula	Meaning
Union	$\max(A(x), B(x))$	Like OR - take higher value
Intersection	$\min(A(x), B(x))$	Like AND - take lower value
Complement	$1 - A(x)$	Like NOT - opposite value
Difference	$\min(A(x), 1 - B(x))$	In A but not strongly in B

Fuzzy Relations

A Fuzzy Relation is created by Cartesian Product of two fuzzy sets. It takes every pair (x, y) from set A and set B and assigns membership = $\min(A[x], B[y])$.

Max-Min Composition

Max-Min Composition combines two fuzzy relations R and S to get a new relation T. For each pair (x, z) :

- Find all possible middle points y in between
- For each middle point y: calculate $\min(R[x,y], S[y,z])$
- Take the maximum of all those minimum values
- That becomes $T[x,z]$

Formula: $T(x,z) = \max \text{ over all } y \text{ of } \min(R(x,y), S(y,z))$

Why MIN first? A chain is only as strong as its weakest link.

Why MAX after? We want the best possible path between two points.

Practical 5: Genetic Algorithm + Neural Network

Problem Statement

Spray drying of coconut milk is a complex process where coconut milk is converted into powder form. We need to find the best combination of Neural Network parameters to get the best quality prediction model. This is an optimization problem solved using Genetic Algorithm.

What is a Neural Network (NN)?

A Neural Network is a computational model inspired by the brain that learns from data. It takes input data, learns patterns, and predicts output. In this practical we use MLPRegressor (Multi Layer Perceptron) from sklearn.

What is a Genetic Algorithm (GA)?

Genetic Algorithm is inspired by natural selection and survival of the fittest. It mimics biological evolution to find optimal solutions.

GA Key Terms

Term	Simple Meaning
Individual	One possible solution
Population	Group of possible solutions
Fitness	How good a solution is (lower MSE = better)
Selection	Pick the best solutions
Crossover	Mix two solutions to create a new one
Mutation	Randomly change a solution slightly
Generation	One round of the entire process

Parameters Being Optimized

Parameter	Range	Meaning
Neurons	10 to 100	Number of neurons in hidden layer of NN
Learning Rate	0.001 to 0.1	How fast the Neural Network learns
Alpha	0.0001 to 0.01	Regularization parameter - prevents overfitting

GA Process Steps

- 1. Initialize random population of GA individuals
- 2. Evaluate fitness of each individual (train NN, calculate MSE)
- 3. Sort by fitness - lowest MSE first (best solution first)
- 4. Select top 50% as parents

- 5. Create children using crossover (mix parameters from two parents)
- 6. Apply mutation (30% chance to randomly change each parameter)
- 7. Replace old population with new population
- 8. Repeat for all generations
- 9. Return best individual found

What is MSE?

Mean Squared Error (MSE) measures how wrong the neural network predictions are. Lower MSE means better model performance. The GA tries to minimize MSE by finding the best NN parameters.

Practical 6: Clonal Selection Algorithm (CSA)

What is Clonal Selection Algorithm?

CSA is a bio-inspired optimization algorithm based on the clonal selection process of B-cells in the immune system. The algorithm was first proposed by de Castro and Timmis in 2002. It uses an iterative process of selection, cloning, and mutation to evolve a population of candidate solutions towards an optimal solution.

Biology vs Computer Analogy

Biology Term	CSA Term	Simple Meaning
Antigen	Problem	What we are trying to solve
Antibody	Solution	One possible answer
Affinity	Fitness	How good a solution is
Cloning	Copying	Make copies of good solutions
Mutation	Changing	Slightly modify copies
Selection	Keeping best	Keep good, remove bad

Key Concepts

1. Antibody Representation

Each solution is called an antibody. Represented as a number in our problem. Example: $x = 0.5$ (solution to minimize x squared).

2. Affinity Maturation

Affinity represents how good a solution is. Formula: $1 / (1 + \text{fitness}(x))$. Higher affinity means better solution. The antibody with x closest to 0 has highest affinity.

3. Cloning

Better solutions get MORE clones. Rank 1 (best) gets most clones. This ensures more exploration around promising solutions.

4. Mutation

Randomly change clones slightly by adding small random value. Helps explore new areas and prevents getting stuck at local optima.

5. Selection

Keep best solutions for next generation. Remove worst solutions. Population size stays constant.

Algorithm Steps

- 1. Initialize random population of antibodies
- 2. Calculate affinity of each antibody

- 3. Select best antibodies for cloning
- 4. Clone them - more clones for better antibodies
- 5. Mutate the clones - add small random changes
- 6. Select best from original + mutated
- 7. Repeat steps 2-6 for specified iterations
- 8. Return best solution found

Problem Being Solved

The code minimizes the function $f(x) = x^2$. The minimum is at $x = 0$ where $x^2 = 0$. Starting from random values between -10 and 10, the algorithm finds x very close to 0.

Practical 7: Artificial Immune Pattern Recognition

Problem Statement

Automatically detect whether a structure (bridge, building, dam) is Healthy (class 0) or Damaged (class 1) using Artificial Immune System (AIS) pattern recognition.

What is AIS?

AIS stands for Artificial Immune System. It is inspired by how our biological immune system detects and fights foreign invaders (viruses, bacteria).

Biology vs AIS Analogy

Biological Immune System	AIS in Computer
Virus/Bacteria (antigen)	Damage pattern in structure
Immune cells (antibodies)	Detectors in algorithm
Detecting virus	Detecting damage
Immune response	Classification result
Mutation of cells	Hypermutation of detectors

What is AIRS?

AIRS = Artificial Immune Recognition System. It is a specific AIS algorithm used for classification problems.

- Randomly select detectors from training data
- Apply hypermutation to detectors (add small random noise)
- For new data, find the closest detector using Euclidean distance
- Assign the label of the closest detector to the new data

What is Euclidean Distance?

Euclidean distance is the straight-line distance between two points. Formula: square root of sum of squared differences. Used to find which detector is closest to a test sample.

What is Hypermutation?

Adding small random noise to detectors using normal distribution. It helps detectors slightly adjust their position to better recognize damage patterns. This simulates the biological mutation of immune cells.

AIS Flow for Structure Damage Detection

- 1. Structure Data collected (Healthy and Damaged structures)
- 2. Feature Extraction - extract relevant features (vibration, stress, strain)

- 3. Data Encoding - convert features to numerical format
- 4. AIS Algorithm (Clonal Selection) trains on data
- 5. Trained Detectors learn damage patterns
- 6. Test Data (unseen structures) is fed to trained detectors
- 7. Damage Detection - classify as healthy or damaged
- 8. Results and Discussion - evaluate accuracy

Practical 8: DEAP - Distributed Evolutionary Algorithms

What is DEAP?

DEAP stands for Distributed Evolutionary Algorithms in Python. It is a framework for implementing genetic algorithms and other evolutionary computation techniques with support for parallelization across multiple processors or computers.

Key Features of DEAP

Feature	Description
Genetic Operators	Built-in crossover (cxBlend), mutation (mutGaussian), selection (selTournament)
Fitness Evaluation	Custom fitness functions - single or multi-objective
Population Management	Initialization methods and selection strategies
Statistics & Logging	Track best fitness, monitor evolution over generations
Parallelization	Multi-core and distributed computing support using multiprocessing
Integration	Works with other Python libraries and frameworks

DEAP Key Components

Component	Purpose
creator	Creates custom fitness and individual classes
base.Toolbox	Registers all GA functions (create, evaluate, mate, mutate, select)
tools	Contains built-in operators like selBest, selTournament, cxBlend, mutGaussian
algorithms	Ready-made GA loops like varAnd

Problem Being Solved

Minimize the function: $f(x) = x_1^2 + x_2^2 + x_3^2$. Answer: $x_1=0, x_2=0, x_3=0$ where $f(0,0,0) = 0$. GA evolves population of 3D individuals to find values close to (0,0,0).

GA Operators Used

Operator	Type	Description
cxBlend	Crossover	Blends values from two parents with $\alpha=0.5$
mutGaussian	Mutation	Adds Gaussian noise with $\mu=0, \sigma=1, \text{indpb}=0.2$
selTournament	Selection	Tournament selection with $\text{tournsize}=3$

Practical 9: Hotel Booking Application using Java RMI

Problem Statement

Design a distributed Hotel Booking System where the Server manages room booking information and the Client can remotely book or cancel rooms for guests using Java RMI.

System Architecture

Component	Role
HotelInterface.java	Remote interface defining bookRoom() and cancelBooking() methods
HotelServer.java	Server that implements interface and manages bookings using HashMap
HotelClient.java	Client that connects to server and calls remote methods
RMI Registry	Name service running on port 1099 to connect client and server

What is RMI Registry?

RMI Registry is a naming service in Java RMI that acts like a Phone Directory. The Server registers its services with a name. The Client looks up that name to find the service. It acts as a middleman between client and server.

Registry Methods

Method	Used By	Purpose
createRegistry(port)	Server	Creates new registry on specified port
getRegistry(host, port)	Client	Connects to existing registry
rebind(name, object)	Server	Registers service (replaces if exists)
lookup(name)	Client	Finds and returns remote service

Client Operations Available

- 1. Book Room - Books a room for a specific guest name
- 2. Cancel Booking - Cancels existing booking for a guest
- 3. Exit - Closes the client application

How bookRoom Works

- Check if guest already has booking using HashMap.containsKey()
- If already booked: return 'Room already booked for [name]'
- If not booked: add to HashMap using put(guestName, true)
- Return 'Room booked successfully for [name]'

How cancelBooking Works

- Check if guest has a booking using `HashMap.containsKey()`
- If no booking found: return 'No booking found for [name]'
- If booking exists: remove from HashMap using `remove(guestName)`
- Return 'Booking cancelled for [name]'

Practical 10: Weather Data Analysis using MapReduce

Problem Statement

Find which year was the hottest and which year was the coolest in India by fetching weather data from the internet and processing it using MapReduce programming model in Python.

System Components

Component	Description
Mapper	Reads weather data, extracts year and temperature, emits (year, temp) pairs
Shuffle & Sort	Groups all temperatures by year using defaultdict
Reducer	Calculates average temperature for each year
Input Data	Weather data from data.gov.in API or hardcoded data
Output	Hottest year and Coolest year with average temperatures

MapReduce Workflow

- 1. Fetch weather data from data.gov.in API (or use hardcoded data)
- 2. MAP PHASE: For each row, extract year and annual temperature as (year, temp) pair
- 3. SHUFFLE & SORT PHASE: Group all temperatures by year using defaultdict
- 4. REDUCE PHASE: Calculate average temperature for each year
- 5. Find hottest year using max() function
- 6. Find coolest year using min() function
- 7. Print final result

Python Libraries Used

Library	Purpose
csv	Read and parse CSV format data
requests	Fetch data from internet API
io.StringIO	Convert string to file-like object for csv.DictReader
collections.defaultdict	Group temperatures by year automatically

What is defaultdict?

defaultdict is a special Python dictionary from collections module. Unlike normal dict that gives KeyError when key not found, defaultdict automatically creates an empty list for any new key. This is used to group temperatures by year without worrying about KeyError.

What is StringIO?

StringIO from io module converts a string into a file-like object. This is needed because csv.DictReader expects a file to read from, but the API returns data as a string. StringIO tricks csv.DictReader into thinking the string is a file.

API Details

The code uses data.gov.in which is Indian Government's open data portal. It provides historical Indian temperature data with fields like YEAR, ANNUAL (annual average temperature), JAN, FEB etc (monthly temperatures). The API requires an API key for authentication.

Sample Output

Field	Value
Hottest Year	2016
Hottest Year Avg Temp	26.20 degrees C
Coollest Year	1917
Coollest Year Avg Temp	24.54 degrees C

--- End of Theory Concepts ---